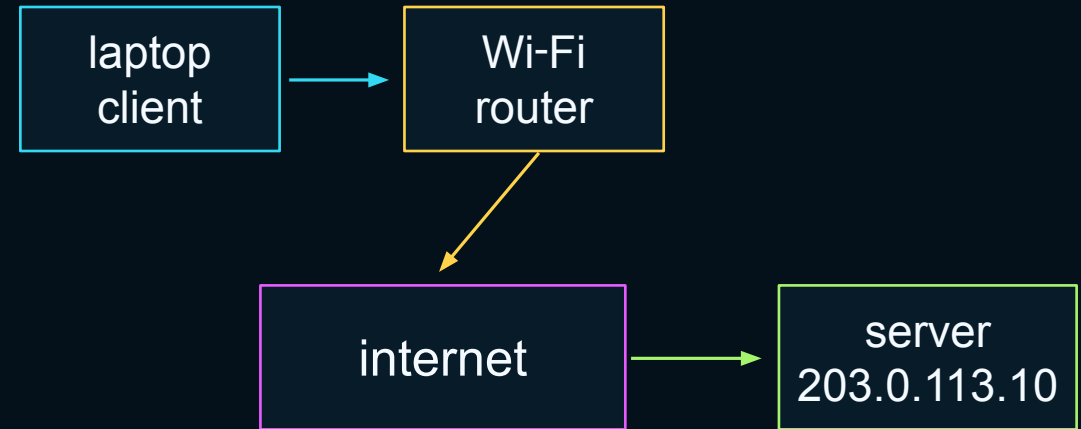


HOW DOES A WEBSITE APPEAR?

Networking and the Internet



GET /

DNS

443

HTML

The internet is not magic. It is organized messages moving between machines.

30 min instruction • 30 min website request card exercise • 30 min morning review

Cold open: one address, many invisible steps

A browser hides a surprising amount of work behind a familiar action.

`https://example.com/reports`



A complete network conversation begins.

Find
the server



Connect
to it



Send
a request



Receive
a response

Today we will trace that path without diving into every engineering detail.

Today's mission

Connect user-friendly internet behavior to plain-language technical parts.

- 1 Recognize** clients, servers, IP addresses, ports, and protocols
- 2 Explain** DNS and website request flow
- 3 Compare** latency, bandwidth, encryption, routers, and firewalls
- 4 Apply** the model to Python clients, servers, and APIs

KEY IDEA

Networking is structured communication.

Machines agree on addresses, rules, and message formats.

By the end, “opening a website” should feel like a traceable sequence, not a black box.

Computers communicate by sending messages

A network lets machines exchange data using agreed rules.



message format
What does it mean?

addressing
Where should it go?

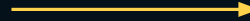
delivery rules
How is it handled?

Local networks and the internet

Your home or office network is one small island connected to many others.

LOCAL NETWORK

Devices near you
Same home, office, classroom, or
base network
Usually behind a router/firewall



INTERNET

Network of networks
Connects many organizations and
providers
Uses public routing between networks

Local does not mean “offline.” It means nearby within your network boundary.

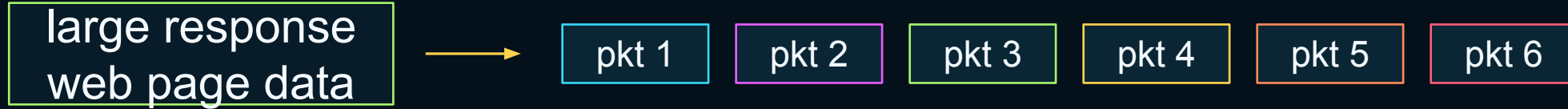
Interfaces, routers, and firewalls

Before data can move, devices need a way out and rules at the boundary.



Packets: small pieces of a larger conversation

Networks usually move data in chunks, not as one giant blob.



Packet header

- source address
- destination address
- ordering / handling details

Packet payload

- piece of the actual data
- part of request or response

Reassembly

- recipient puts pieces together
- missing pieces can be retried

The user sees “a page.” The network may carry many packets.

IP addresses: where should the packets go?

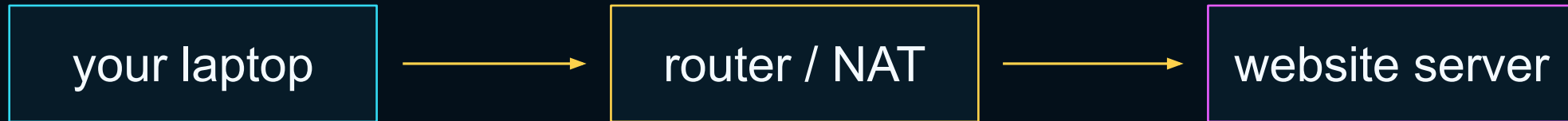
An IP address is a network location identifier.

192.168.1.25

private IP address

203.0.113.10

public IP address



Private addresses work inside a local network. Public addresses are reachable across the internet.

Domain names and DNS

Humans prefer names. Networks route to addresses. DNS connects the two.



Domain name
Easy for people to
remember

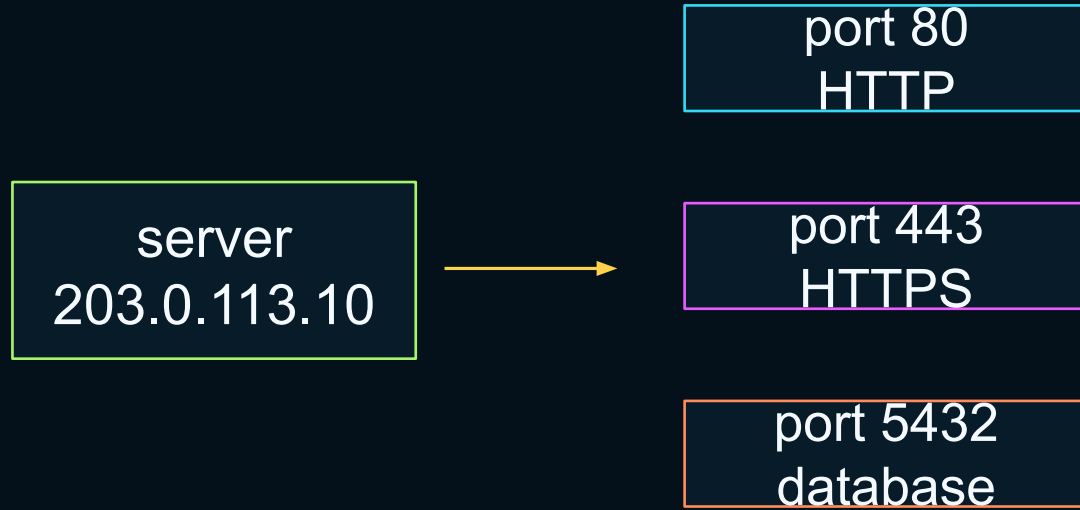
DNS
Directory-style system

IP address
Useful for routing
packets

DNS is why you can type a name instead of memorizing a server address.

Ports: which service on that machine?

An IP address gets you to the building; a port gets you to the right counter.



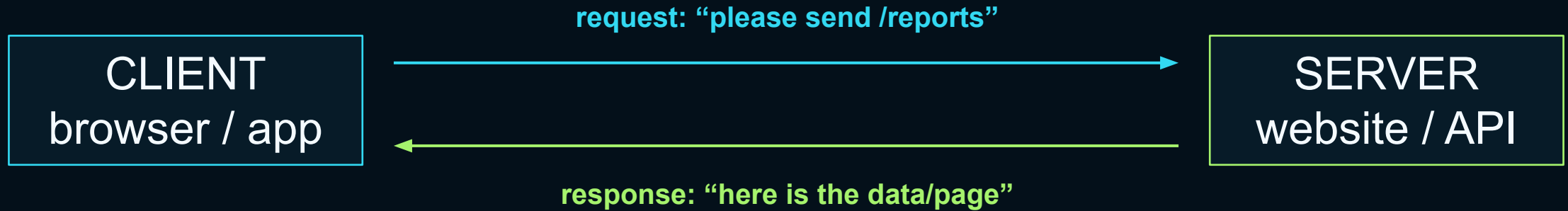
IP address says:
“Which machine?”

Port says:
“Which network service on
that machine?”

`https://example.com:443/reports`

Clients and servers: request and response

Many internet interactions follow a simple conversation pattern.



Client
starts the conversation

Server
listens and responds

Protocol
defines the rules

Protocols: agreed rules for communication

A protocol defines how messages should be formatted and handled.

HTTP

web requests and
responses

HTTPS

HTTP protected by
encryption

Other protocols
email, DNS,
databases, file
transfer

```
GET /reports HTTP/1.1  
Host: example.com  
Accept: text/html
```

```
HTTP/1.1 200 OK  
Content-Type: text/html  
<html>...</html>
```

Protocols let independent systems understand each other.

Encryption at a high level

HTTPS helps protect the conversation from casual reading or tampering.



Without protection
Someone on the path
may read or modify
data.

With HTTPS
The conversation is
encrypted and
authenticated.

Still important
The website itself must
also be trustworthy.

HTTPS does not mean “the site is good.” It means the connection is protected.

Latency and bandwidth

Two different reasons a network can feel slow.

LATENCY

How long it takes for a message
to make the trip
Feels like delay or lag
“Are we there yet?”

small request + high latency
= annoying pause

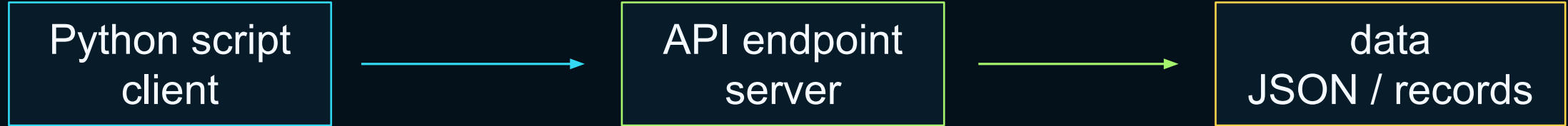
BANDWIDTH

How much data can move per
second
Feels like download capacity
“How wide is the pipe?”

large video + low bandwidth =
slow loading

APIs: structured network communication

An API is a planned way for software to request or send data.



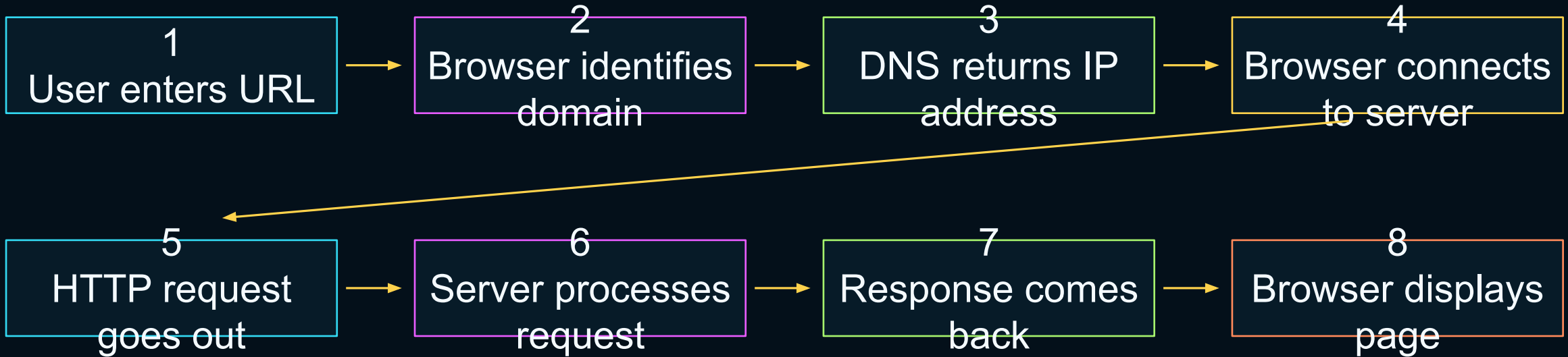
```
GET /api/orders?date=2026-07-17  
Authorization: Bearer ...
```

```
{  
  "orders": 128,  
  "total": 4200.75  
}
```

Later in Python, you can use APIs to automate work between systems.

Simplified website request

A browser turns one user action into a sequence of technical steps.



This is the sequence students will arrange in the guided exercise.

Connection to future Python work

Python programs can participate in network conversations.

CLIENT

request data from
websites, APIs,
databases, or services

SERVER

receive requests and
send responses

AUTOMATION

move data between
systems reliably

```
# Python as a client  
response = requests.get(url)
```

```
# Python as a server  
@app.get("/orders")
```

Networking turns Python from “local script” into “software that talks to other systems.”

EXERCISE

Guided exercise: website request cards

Arrange the cards into a reasonable order. Then label the parts.

User enters a URL

Browser requests
an IP address

DNS returns an
address

Browser opens a
connection

Browser sends an
HTTP request

Server processes
the request

Server sends a
response

Browser displays
the page

After ordering the cards, identify: client, server, address, protocol, request, response.

Work in pairs. Focus on the story, not perfect network engineering.

EXERCISE

Suggested answer: website request sequence

The exact details can vary, but this is the core mental model.

- 1 User enters a URL
- 2 Browser identifies domain
- 3 Browser asks DNS for IP
- 4 DNS returns address
- 5 Browser connects to server
- 6 Browser sends HTTP request
- 7 Server sends response
- 8 Browser displays page

Browser = client. Website machine/application = server.

EXERCISE

Identify the parts

Use the same website scenario and label the nouns.

Client

Browser on the user's
computer

Server

Computer/app
hosting the site

Address

Domain name + IP
address

Protocol

HTTP or HTTPS

Request

What the client asks
for

Response

What the server
returns

Example: GET `https://example.com/reports`

Response: status code + headers + page/data

Discussion: what changed from “magic” to “traceable”?

Use plain language. Technical precision can come later.

What step finds the destination machine?

Why do we need both an IP address and a port?

What is the difference between a request and a response?

Why does HTTPS matter?

Where could Python fit into this conversation?

Suggested pacing: 5–7 minutes, then move to morning applied review.

Morning applied challenge and review

Now connect Blocks 1–4 into two complete stories.

SCENARIO 1

Opening a spreadsheet

Storage → filesystem →
memory → processor →
display

SCENARIO 2

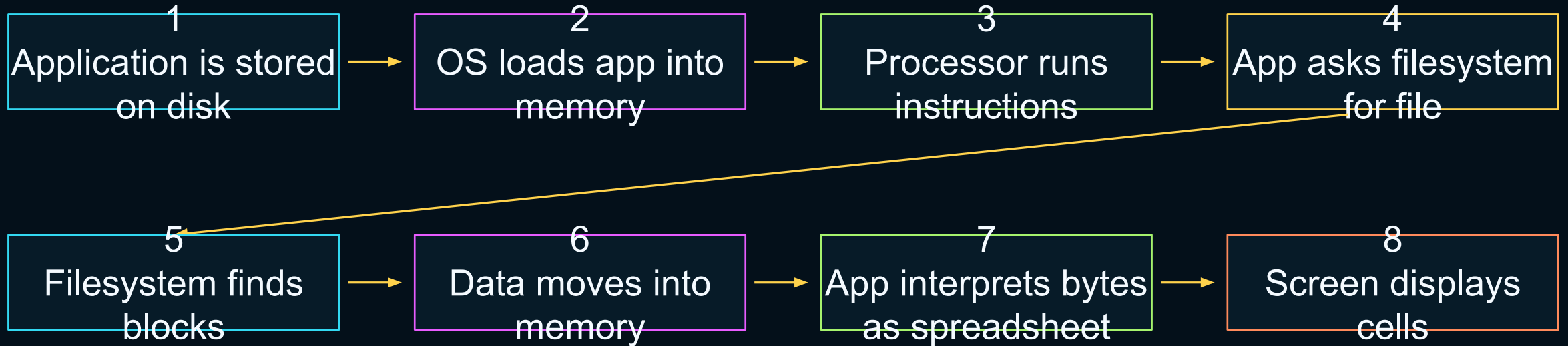
Opening a website

DNS → connection → request
→ server → response →
browser

Goal: explain each scenario in plain English without hiding behind buzzwords.

Scenario 1: opening a spreadsheet

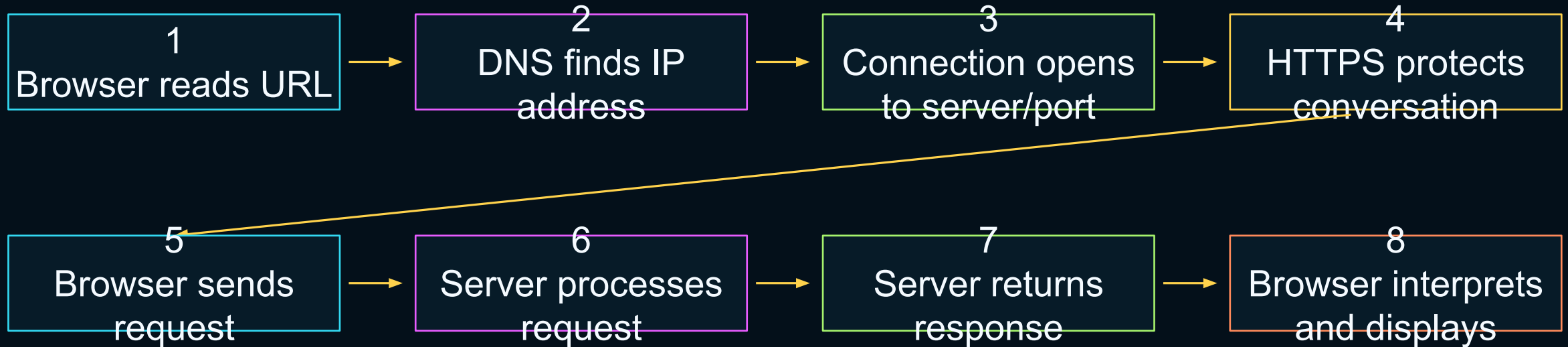
Trace the journey from saved file to visible data.



Key connection: the file name points to metadata, metadata points to blocks, and the application interprets the bytes.

Scenario 2: opening a website

Trace the journey from address bar to displayed page.



Key connection: the browser is a client, the website is served by a server, and HTTP/HTTPS structures the conversation.

REVIEW

Morning knowledge check: foundation terms

Students should be able to explain these without reading definitions.

bit
0 or 1

byte
8 bits

machine code
processor
instructions

interpreter
runs high-level
code

memory
running
workspace

storage
saved for later

filesystem
tracks files

IP address
network location

port
service endpoint

Challenge: explain all nine to a non-technical coworker in two minutes.

Morning knowledge check: client and server

A final plain-language definition before the AM wrap.

CLIENT

The software or machine that starts a request.

Examples: browser, phone app,
Python script.

SERVER

The software or machine that listens for requests and responds.

Examples: website backend,
API, database service.

request

response

Wrap-up discussion

Use these questions to close Block 4 and the morning sequence.

How does DNS help a browser find a server?

Why is a port needed after the IP address is known?

What work does a browser perform after receiving a response?

How could Python act as a client? How could it act as a server?

Where do bits, memory, filesystems, and networking all meet in real software?

Next up: we start turning these mental models into actual Python code.